Diagramming - Use Cases

After we've done our **preliminary information** gathering *(we may need to go back depending on issues, or if we use Agile Development.)* We can draw HIGH LEVEL diagrams to help both the *technical side* (Us Developers) and the *business side* (client) **understand what's going on in the system.**

Some of the diagrams we can draw are USE CASE diagrams & ACTIVITY diagrams; both try to explain *what scenarios* **the system can complete.**

- But what is a use case? Simply, **it's an scenario/action that can be done under the system**; *User check out book, user check in book, Librarian Revoke Fine, User Pay Fine.*

Use Case Diagram: What this diagram does is; show EVERY scenario the system can go through, and *which people interact with those scenarios*.

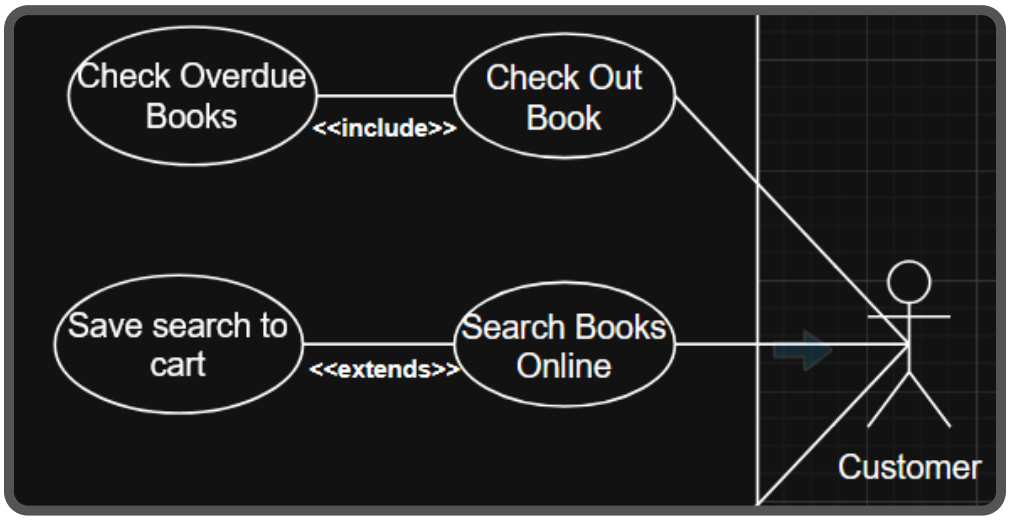
- Actors: The Librarian, The Customer, The Admin *(Lines are drawn to which cases they interact with.)*
- Cases: The scenarios; i.e. *Rent a book, Pay a fine, Revoke Fine*

We have **the cases**, and we have **the people who interact with them**; as simple as it gets.

There are also TWO special types:

- Include, and Extend *(on next page)*

- These diagrams *(Use Case Diagram)* Have 2 special lines,
  - **<<include>>** This is a scenario that always happens alongside another, *i.e. when a Customer attempts to checkout, it must INCLUDE "check for overdue fines"*
  - **<<extends>>** This is an optional scenario that MAY happen alongside another i.e when a customer is searching, they can "*Add to cart*".



Activity Diagram: This takes one of those CASES (scenarios), and *goes into depth*. It creates the step by step algorithm for that one; Let's do "**Check Out Book**"

- Let's say *from our information gathering*; **we know the normal process for Checking out a book is**; The user *searches for a book*, goes to checkout, the system checks if the *user has any overdue charges*, if so user can NOT check out, if they don't they can continue to checkout; enter their details; optionally decide to check out another book; if not finish check out.

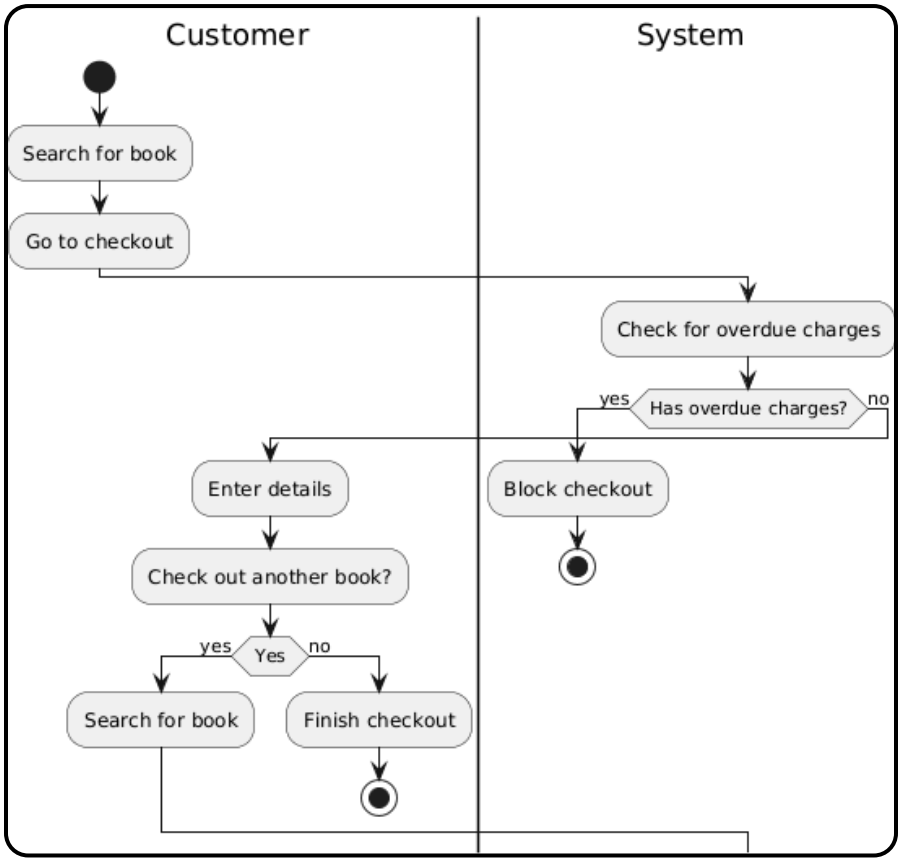
For **Activity Diagrams** we follow a syntax; there are separated boxes for each actor, steps in the algorithm are notated **by an oval.** Choices are under "**decision diamonds**" which is, just a diamond with 2 paths (yes or no)

- Then there's the start at the end
  - Notated by **O→** for start
  - Notated by **→O** for end.

We will split our diagram into TWO sections; **Customer & System.** And *analyse*

- The check for *Overdue Charges* is done AUTOMATICALLY, so it is under the system; it is also a YES/NO, thus it will be under a **decision diamond.**

- The check for *Check Out Other Books*, is done via the customer, so it a under the customer; it is also a YES/NO, thus it will be a **decision diamond.**



We have TWO parts to the case, an ACTOR (customer) talks to a system, *this is normally how it works*. Sometimes you may have more actors, (like the Librarian, then the Customer, then the system. But for now we're gonna stick with 2, **the final part of the lab will cover this.**)

That's the **general concept for diagramming** for USE CASES; *our next part will cover more computer science oriented diagramming*;

- Object & Class Diagramming.